# RealMan Robotic Arm ROS2 Package Overview V1.7.0

RealMan Intelligent Technology (Beijing) Co., Ltd.

Version: V1.7.0

The package is mainly used for providing ROS2 support for the robotic arm, and the following is the use environment.

- Currently supported robotic arms include RM65, RM75, ECO62, ECO63, ECO65, RML63, GEN72, and RX75 dual-arm series. For details, refer to RealMan robots (http://www.realman-robotics.com/).
- Version V1.7.0.
- Based on robotic arm controller version 1.7.3.
- The Ubuntu version is 22.04.

The following is the installation and use tutorial of the package.

### 1\. Build the environment

Before using the package, we first need to do the following operations.

- 1.Install ROS2
- 2.Install Moveit2
- 3.Configure the package environment
- 4.Compile

### 1.Install_ROS2

We provide the installation script for ROS2, ros2_install.sh, which is located in the scripts folder of the rm_install package. In practice, we need to move to the path and execute the following commands.

```
sudo bash ros2_install.sh
```

If you do not want to use the script installation, you can also refer to the website ROS2_INSTALL (https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debians.html).

### Install_Moveit2

We provide the installation script for Moveit2, moveit2_install.sh, which is located in the scripts folder of the rm_install package. In practice, we need to move to the path and execute the following commands.

```
sudo bash moveit2_install.sh
```

If you do not want to use the script installation, you can also refer to the website Moveit2_INSTALL (https://moveit.ros.org/install-moveit2/binary/).

### Configure_the_package_environment

This script is located in the lib folder of the rm_driver package. In practice, we need to move to the path and execute the following commands.

```
sudo bash lib_install.sh
```

### Compile

After the above execution is successful, execute the following commands to compile the package. First, we need to build a workspace and import the package file into the src folder under the workspace, and then use the colcon build command to compile.

```
mkdir -p ~/ros2_ws/src
cp -r ros2_rm_robot ~/ros2_ws/src
cd ~/ros2_ws
colcon build --packages-select rm_ros_interfaces
source ./install/setup.bash
colcon build
```

After the compilation is completed, the package can be run.

## 2\. Function running

Package introduction

1. 1. Installation and environment configuration (rm_install (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_install))

- This package is the auxiliary package for using the robotic arm. It is mainly used to introduce the installation and construction method of the package use environment, the installation of the dependent library and the compilation method of the function package.

2. 3. Hardware driver (rm_driver (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_driver))

- This package is the ROS2 underlying driver package of the robotic arm. It is used to subscribe and publish the underlying related topic information of the robotic arm.

3. 5. Launch (rm_bringup (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_bringup))

- This package is the node launch package of the robotic arm. It is used to quickly launch the multi-node compound robotic arm function.

4. 6. Model description (rm_description (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_description))

- This package is the model description package of the robotic arm. It is used to provide the robotic arm model file and model load node, and provide the coordinate transformation relationship between the joints of the robotic arm for other packages.

5. 7. ROS message interface (rm_ros_interfaces (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_ros_interfaces))

- This package is the message file package of the robotic arm. It is used to provide all control messages and state messages for the robotic arm to adapt to ROS2.

6. 8. MoveIt2 control programs (rm_moveit2 (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_moveit2))

- This package provides MoveIt2-based control examples for both real and simulated robots, including joint-space motion, pose goals, and Cartesian trajectories.

7. 9. Moveit2 configuration (rm_moveit2_config (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_moveit2_config))

- This package is the moveit2 adaptation package of the robotic arm. It is used to adapt and realize the moveit2 planning and control functions of various series of robotic arms, mainly including the control functions of virtual robotic arm control and real robotic arm control.

8. 10. Moveit2 and hardware driver communication connection (rm_config (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_control))

- This package is the communication connection package between the underlying driver package (rm_driver) and the moveit2 package (rm_moveit2_config). It is mainly used to subdivide the planning points of moveit2 and then pass them to the underlying driver package in the form of transmission to control the motion of the robotic arm.

9. 11. Gazebo simulation robotic arm control (rm_gazebo (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_gazebo))

- This package is the gazebo simulation robotic arm package. It is mainly used to display the robotic arm model in the gazebo simulation environment, and the planning and control of the simulated robotic arm can be carried out through moveit2.

10. 12. Use examples (rm_examples (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_example))

- This package is some use examples of the robotic arm, and it is mainly used to realize some basic control functions and motion functions of the robotic arm.

11. 13. Technical documentation (rm_docs (https://github.com/RealManRobot/ros2_rm_robot/tree/humble/rm_doc))

- This package is an introduction document package, which mainly includes a document that provides an overall introduction to the content and usage of the packages, as well as a document that provides a detailed introduction to the content and usage of each package.

The above are the current major packages. Each package has its own role. For more details, refer to the documentation under `rm_doc`.

### 2.1 Run the virtual robotic arm

Use the following command to launch the gazebo to display the simulation robotic arm, and launch moveit2 for the planning and control of the simulation robotic arm.

```
source ~/ros2_ws/install/setup.bash
ros2 launch rm_bringup rm_<arm_type>_gazebo.launch.py
```

\ should be replaced with the actual arm type, such as 65, 75, eco62, eco63, eco65, 63, 63_III, gen72, or gen72_II. For RX75, use `rm_rx75_6fb_gazebo.launch.py` for RX75-6FB and `rm_rx75_6fb_v_gazebo.launch.py` for RX75-6FB-V. For example, when using an RM65 robotic arm, the command is as follows.

```
ros2 launch rm_bringup rm_65_gazebo.launch.py
```

After successful launch, you can use moveit2 for the control of the virtual robotic arm.

### 2.2 Control the real robotic arm

Use the following command to launch the hardware driver of the robotic arm, and launch moveit2 for the planning and control of the robotic arm.

```
source ~/ros2_ws/install/setup.bash
ros2 launch rm_bringup rm_<arm_type>_bringup.launch.py
```

\ should be replaced with the actual arm type, such as 65, 75, eco62, eco63, eco65, 63, 63_III, gen72, or gen72_II. For RX75, use `rm_rx75_6fb_bringup.launch.py` for RX75-6FB and `rm_rx75_6fb_v_bringup.launch.py` for RX75-6FB-V. For example, when using an RM65 robotic arm, the command is as follows.

```
ros2 launch rm_bringup rm_65_bringup.launch.py
```

After successful launch, you can use moveit2 for the control of the real robotic arm.

### Safety Tips

Please refer to the following operation specifications when using the robotic arm to ensure the user's safety.

- Check the installation of the robotic arm before each use, including whether the mounting screw is loose and whether the robotic arm is vibrating or trembling.
- During the running of the robotic arm, no person shall be in the falling or working range of the robotic arm, nor shall any other object be placed in the robot arm's safety range.
- Place the robotic arm in a safe location when not in use to avoid it from falling down and damaging or injuring other objects during vibration.
- Disconnect the robotic arm from the power supply in time when not in use.